

**ADNEKO**

# NEXUS COMMIT

## FASE 1 - ESTADO REAL

---

Qué es · Para qué sirve · Qué funciona hoy · Qué falta para verlo operar

### OBJETIVO DE ESTE DOCUMENTO

Explicar NEXUS COMMIT sin lenguaje abstracto. Este documento separa lo que está realmente implementado y probado de lo que todavía falta para que una persona pueda abrir una pantalla, usar el sistema y verlo funcionar de principio a fin.

**Fecha de corte: 22 de septiembre de 2026**

Repositorio: siandeda2-beep/adneko-nexus-commit

## 1. RESPUESTA CORTA: ¿QUÉ ES NEXUS COMMIT?

### NEXUS COMMIT ES UN MOTOR DE CONTROL DE EJECUCIÓN DE COMPROMISOS

Su función es recibir un compromiso que tiene una ventana de tiempo, una autoridad y una acción real asociada; controlar si se responde a tiempo; resolver qué ocurre si nadie responde; ejecutar la acción por medio de un proveedor externo; y guardar evidencia durable de todo el proceso. Si el proveedor devuelve un resultado incierto, NEXUS COMMIT no inventa el resultado: entra en reconciliación hasta poder determinar SUCCEEDED, FAILED o INDETERMINATE.

**No es:** un CRM, un gestor de tareas, un dashboard, un chatbot, una IA conversacional ni un PLC de seguridad.

**Sí es:** infraestructura de software para que una decisión con plazo y consecuencias operativas no dependa de memoria humana, supuestos o resultados ambiguos.

### Ejemplo sencillo

1. Una operación necesita una confirmación antes de una hora límite.
2. NEXUS COMMIT registra el compromiso, la ventana de decisión, el actor y la acción que debe ejecutarse.
3. Si la persona confirma a tiempo, NEXUS COMMIT autoriza la ejecución.
4. Si la persona rechaza, el compromiso se resuelve como rechazado.
5. Si nadie responde y vence el tiempo, NEXUS COMMIT aplica la regla de fallback definida; no decide arbitrariamente.
6. Cuando hay que ejecutar, NEXUS COMMIT llama al proveedor externo con una identidad e idempotency key durables.
7. Si el proveedor responde UNKNOWN, NEXUS COMMIT no repite a ciegas ni declara éxito: entra en reconciliación.
8. Cuando el estado remoto se confirma, finaliza SUCCEEDED/FAILED/INDETERMINATE y deja evidencia en PostgreSQL y eventos durables.

### EN UNA FRASE PARA UN INVERSIONISTA

“NEXUS COMMIT convierte compromisos con tiempo y consecuencias reales en ejecuciones controladas, auditables y reconciliables, evitando decisiones fuera de ventana, dobles ejecuciones y falsos estados de éxito.”

## 2. ¿QUÉ SE HACE CON ESTE SISTEMA?

Un sistema externo, operador o aplicación utiliza NEXUS COMMIT para cuatro operaciones públicas de FASE 1:

| Operación          | Para qué sirve                                                                                | Estado                                     |
|--------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------|
| CREATE_COMMITMENT  | Crear un compromiso con actor, recursos, ventana de decisión, fallback y acción de ejecución. | Implementado                               |
| RESPOND_COMMITMENT | Confirmar o rechazar el compromiso dentro de las reglas de tiempo y autoridad.                | Implementado                               |
| QUERY_STATE        | Consultar el estado actual y resultado del compromiso.                                        | Implementado                               |
| QUERY_RELIABILITY  | Consultar métricas observables de confiabilidad del actor (RR, TR, MRT).                      | Implementado; score compuesto no inventado |

**Entrada real hoy:** HTTP API: POST /v1/commands y ruta de compatibilidad POST /adneko/v1/command.

**Procesamiento real:** API + PostgreSQL + Worker + Execution Provider + Reconciliation + Outbox.

**Lo que falta para que usted “lo vea” sin terminal:** una interfaz/demostrador que convierta estas APIs y estados en botones, formularios y una línea de tiempo visible.

### FLUJO OPERATIVO

|                  |                  |                                |                    |                        |           |                    |
|------------------|------------------|--------------------------------|--------------------|------------------------|-----------|--------------------|
| Crear compromiso | Esperar decisión | Confirmar / Rechazar / Timeout | Ejecutar proveedor | Reconciliar si UNKNOWN | Finalizar | Publicar evidencia |
|------------------|------------------|--------------------------------|--------------------|------------------------|-----------|--------------------|

### 3. ESTADO REAL DE FASE 1 - QUÉ ESTÁ HECHO

Semáforo usado en este documento: **VERDE** = implementado/probado. **AMARILLO** = implementado pero falta empaquetarlo/desplegarlo de forma visible. **ROJO** = todavía no existe como producto visible/comercial.

| Componente                   | Estado real | Evidencia actual                                                                              | Qué falta                                                                              |
|------------------------------|-------------|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Contratos y dominio          | VERDE       | Contratos v1, reglas temporales, estados, fallback, invariantes.                              | Nada crítico para demo.                                                                |
| API HTTP                     | VERDE       | POST /v1/commands, POST /adneko/v1/command, /health/live, /health/ready.                      | Desplegar y conectar a una UI demo.                                                    |
| Autenticación + autoridad    | VERDE       | Principal autenticado separado de autoridad de negocio; grants tenant/scope.                  | Provisionar un tenant/principal de demo.                                               |
| PostgreSQL + RLS             | VERDE       | 38 migraciones, FORCE RLS, runtime restringido, triggers de autoridad.                        | Nada crítico para demo; usar BD certificación/staging.                                 |
| Execution Dispatcher         | VERDE       | EXECUTION_PENDING -> EXECUTING; creación de job y evidencia.                                  | Exponer visualmente el estado.                                                         |
| Provider Journal             | VERDE       | EXECUTE/STATUS_QUERY STARTED -> COMPLETED -> APPLIED; identidad durable.                      | Usar proveedor simulado/real en demo.                                                  |
| Reconciliación               | VERDE       | UNKNOWN -> WAITING_RECONCILIATION -> queryStatus -> resultado final.                          | Exponer timeline visible.                                                              |
| Retry seguro                 | VERDE       | Replay solo cuando es seguro/idempotente; no duplica efecto a ciegas.                         | Probar caso visible opcional.                                                          |
| Crash recovery               | VERDE       | Attempts obsoletos detectables; EXECUTE ambiguo -> INDETERMINATE; STATUS_QUERY reconsultable. | No necesario para demo básica; sí para release.                                        |
| Outbox                       | VERDE       | Eventos durables, entrega at-least-once, terminal FAILED visible en health.                   | Conectar endpoint real o publisher de demo.                                            |
| Readiness/health             | VERDE       | API/worker exigen runtime restringido + migración 038; health operacional.                    | Activar thresholds adecuados en staging.                                               |
| Reliability observada        | VERDE       | RR, TR, MRT por actor/tipo/ventana.                                                           | DC y score compuesto siguen no definidos por fuente.                                   |
| Certificación formal         | AMARILLO    | pnpm certify, 12 integration suites + persistence proof + runtime demo.                       | Ejecutar certificación completa en un runner/local con credenciales del commit exacto. |
| Docker / despliegue          | AMARILLO    | Targets API, worker y migrator; contrato de rollout documentado.                              | Levantar staging accesible.                                                            |
| Dashboard / interfaz visible | ROJO        | No existe dashboard productivo certificado.                                                   | Construir una UI mínima de demo/operación.                                             |
| Demo “abrir y usar”          | ROJO        | No hay hoy un enlace donde usted entre y presione botones.                                    | Empaquetar API+worker+provider simulator+UI en un entorno de demo.                     |
| Despliegue comercial público | ROJO        | No declarado.                                                                                 | Staging/producción, dominio/TLS, secretos, observabilidad, backup y soporte.           |

## 4. ¿QUÉ FUNCIONA DE VERDAD HOY? - EVIDENCIA

### PRUEBA REAL EJECUTADA CONTRA POSTGRESQL

Se ejecutó una transacción real sobre la base de certificación. Resultado: lifecycle FINALIZED, decision CONFIRMED, execution SUCCEEDED, job SUCCEEDED, provider\_execution\_id remote-visible-42 y provider\_authority\_id visible-demo-authority-v1. La transacción terminó con ROLLBACK para no dejar residuos.

#### Intentos durables comprobados:

- EXECUTE -> result UNKNOWN -> journal COMPLETED -> application APPLIED.
- STATUS\_QUERY -> result SUCCEEDED -> journal COMPLETED -> application APPLIED.

#### Eventos persistidos en orden:

- EXECUTION\_REQUESTED
- EXECUTION\_RECONCILIATION\_REQUIRED
- EXECUTION\_RECONCILED

#### Cadena observada:

EXECUTION\_PENDING -> EXECUTION\_REQUESTED -> EXECUTE / UNKNOWN ->  
WAITING\_RECONCILIATION -> STATUS\_QUERY / SUCCEEDED -> FINALIZED / SUCCEEDED

También está implementada en el repositorio la demo runtime PostgreSQL: `pnpm demo:runtime-postgres`

- Valida principal runtime restringido.
- Valida readiness API y worker con schema 038.
- Ejecuta dispatch -> provider UNKNOWN -> reconciliation -> SUCCEEDED.
- Entrega 3 eventos por Outbox.
- Exige 0 violaciones de operational health.
- Repite dispatch/reconciliation/outbox y exige 0 trabajo adicional (replay safe).
- Limpia las filas del tenant de demo.

### IMPORTANTE

Esto demuestra que el motor y la persistencia funcionan. No demuestra todavía una experiencia de usuario final. Usted no ve un dashboard porque ese dashboard no existe hoy como producto certificado.

## 5. TODO LO DESARROLLADO EN FASE 1 - POR BLOQUE

### A. Core de compromiso y decisión

- Contrato canónico de comandos y eventos.
- Máquina de estados del compromiso.
- Ventanas FIXED/ROLLING y uso de tiempo autoritativo.
- Confirmación, rechazo y timeout.
- Fallback determinista ante ausencia de respuesta.
- Reserva/capacidad de recursos antes de abrir autoridad de decisión.

### B. Ejecución externa

- Execution intent durable.
- Execution jobs persistentes.
- Execution provider registry y capacidades.
- Idempotency key durable.
- Estados SUCCEEDED/FAILED/UNKNOWN/INDETERMINATE.
- Reglas de retry seguro y prohibición de replay inseguro.

### C. Journal y recuperación

- Journal de EXECUTE y STATUS\_QUERY.
- STARTED/COMPLETED/APPLIED/ABANDONED.
- Request fingerprint.
- Provider authority durable.
- Recuperación conservadora tras crash.
- Evidencia de intentos obsoletos y aplicación pendiente.

### D. Reconciliación

- Reconciliation worker con lease.
- queryStatus al provider.
- Manejo RUNNING/SUCCEEDED/FAILED/UNKNOWN.
- Máximo de reconciliaciones.
- Finalización conservadora como INDETERMINATE cuando no puede probarse el resultado.

### E. Persistencia y aislamiento

- PostgreSQL Event Store.
- commitments\_view.
- execution\_jobs / execution\_attempts.
- Outbox durable.
- FORCE RLS multi-tenant.
- Roles runtime/migrator/authority separados.
- 38 migraciones numeradas, inmutables y con checksums.

### F. Autoridad y seguridad operacional

- Autenticación separada de autorización.
- Authority grants tenant/scope.
- Principal runtime restringido.
- Triggers de integridad/estado/autoridad.
- Endpoints de provider con HTTPS y rechazo de localhost/IPs no públicas explícitas.
- Bearer token confinado al mismo origin execute/status.

## G. Workers y operaciones

- Activation recovery.
- Deadline processing.
- Execution dispatch.
- Execution retry.
- Provider-call recovery.
- Reconciliation.
- Outbox delivery.
- Leases y FOR UPDATE SKIP LOCKED.

## H. Readiness y observabilidad

- /health/live y /health/ready.
- Schema floor 038.
- Backlogs de created/deadline/dispatch/retry/reconciliation/outbox.
- Detección de DEADLINE\_FAILED y OUTBOX\_FAILED.
- Detección de RUNNING no autorizado, stale attempts y application backlog.
- Operational health thresholds.

## I. Reliability observada

- QUERY\_RELIABILITY.
- Response Rate (RR).
- Timeout Rate (TR).
- Median Response Time (MRT).
- Aislamiento por tenant y authority reliability:read.
- Decision Consistency y score compuesto deliberadamente no inventados hasta definir fórmula fuente.

## J. Build, despliegue y certificación

- Toolchain Node.js 22 + pnpm 10.15.1.
- Docker targets: api, worker, migrator.
- Migrator con advisory lock + checksums.
- pnpm certify como endpoint formal.
- 12 suites integration declaradas explícitamente.
- Persistence atomicity/rollback proof.
- Runtime PostgreSQL proof.
- CI configurada para delegar certificación a pnpm certify.

## K. Documentación

- Deployment contract.
- Authority / reliability / ADNEKO integration docs.
- Manual de transferencia técnica, FAT/SAT, operación/mantenimiento y dossier de evidencia creados para FASE 1.
- BOM de despliegue, mapa I/O/red, configuración y criterios de aceptación.

## 6. QUÉ FALTA PARA QUE USTED PUEDA ABRIRLO Y VERLO FUNCIONAR

### ESTE ES EL GAP PRINCIPAL

El motor existe; lo que falta es empaquetar ese motor en una demostración visible y operable. Hoy el valor está detrás de APIs, workers, PostgreSQL y scripts. Para una presentación a un inversionista hace falta convertirlo en una experiencia de 1 clic / 1 pantalla.

| Pendiente                          | Qué significa                                                                                                                      | Prioridad                           |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| 1. Entorno de demo/staging         | Levantar PostgreSQL, migraciones, API y worker en una URL/entorno estable.                                                         | Imprescindible                      |
| 2. Tenant + autoridad de demo      | Crear un tenant, principal y grants controlados para que API acepte comandos reales.                                               | Imprescindible                      |
| 3. Provider simulator              | Servicio de demo que acepte EXECUTE y STATUS_QUERY y permita simular SUCCEEDED, FAILED y UNKNOWN->SUCCEEDED.                       | Imprescindible                      |
| 4. Interfaz visual mínima          | Pantalla con: Crear compromiso, Confirmar/Rechazar, dejar Timeout, Ver estado, Ver execution timeline, Ver eventos.                | Imprescindible para “verlo”         |
| 5. Orquestador de demo             | Un comando o botón que arranque API + worker + provider simulator + UI y verifique health.                                         | Muy recomendable                    |
| 6. Flujo API end-to-end visible    | La UI debe enviar CREATE_COMMITMENT por la API; no insertar filas manualmente.                                                     | Imprescindible                      |
| 7. Vista de evidencia              | Mostrar commitment state, job, attempts, provider authority, eventos y outbox.                                                     | Imprescindible para inversionista   |
| 8. Certificación del commit exacto | Ejecutar pnpm certify completo con credenciales de integración y conservar PASS asociado al commit.                                | Release gate                        |
| 9. CI ejecutando pasos             | GitHub Actions hoy ha fallado antes de Checkout en runs recientes; hace falta un runner/CI que llegue a ejecutar la certificación. | Release gate                        |
| 10. Despliegue comercial           | Dominio/TLS, secretos, backups, monitorización, alertas, política de actualización y soporte.                                      | Necesario para producción comercial |

**Criterio mínimo de “YA LO PUEDO VER FUNCIONAR”:** desde un navegador, crear un compromiso por API, verlo pasar por sus estados, ejecutar un provider simulado, forzar un UNKNOWN, observar reconciliación y terminar SUCCEEDED con la evidencia visible; sin tocar SQL manualmente.

## 7. QUÉ PUEDE DECIRSE HOY A UN INVERSIONISTA

### VERSIÓN CORRECTA, SIN EXAGERAR

“NEXUS COMMIT tiene un core software implementado para controlar compromisos temporales, autoridad, ejecución externa, resultados ambiguos, reconciliación y evidencia durable. La ruta central ha sido validada contra PostgreSQL real y el repositorio contiene API, workers, migraciones, aislamiento RLS, outbox, readiness y un harness formal de certificación. Lo que falta no es inventar el motor: falta desplegarlo en un entorno de demo, añadir una interfaz visible y completar una certificación/release del commit exacto para mostrarlo como producto operable.”

#### Lo que NO debe decirse todavía:

- “Está en producción comercial.”
- “Tiene dashboard terminado.”
- “El commit actual tiene CI verde.”
- “Puede entregarse hoy a un cliente no técnico y operar con botones.”
- “Todo el producto está certificado para un entorno específico.”

#### Lo que SÍ puede demostrarse con evidencia:

- Repositorio y arquitectura del core.
- 38 migraciones y controles de autoridad/aislamiento.
- Prueba PostgreSQL real terminando FINALIZED/SUCCEEDED.
- Execution journal con EXECUTE UNKNOWN y STATUS\_QUERY SUCCEEDED.
- Eventos de reconciliación durables.
- Scripts de demo y certificación formal.
- Documentación de despliegue, operación y transferencia técnica.

## 8. CAMINO DIRECTO DESDE HOY HASTA “ABRO Y LO VEO FUNCIONAR”

| Paso | Resultado esperado            | Definición clara                                                           |
|------|-------------------------------|----------------------------------------------------------------------------|
| A    | Congelar el commit de demo    | Partir de main oficial y no mezclar nuevos features.                       |
| B    | Preparar staging              | PostgreSQL + migrator + roles runtime/authority + API + worker.            |
| C    | Crear provider simulator      | Endpoints execute/status con escenarios controlados.                       |
| D    | Provisionar demo              | Tenant, principal y authority grants.                                      |
| E    | Construir UI mínima           | Crear compromiso, responder, ver timeline, ver attempts/eventos.           |
| F    | Conectar UI a API real        | Todo entra por POST /v1/commands; nada por SQL manual.                     |
| G    | Ejecutar 3 historias visibles | CONFIRM->SUCCEEDED; TIMEOUT->fallback; UNKNOWN->reconciliation->SUCCEEDED. |
| H    | Ejecutar pnpm certify         | Guardar salida y commit exacto.                                            |
| I    | Grabar evidencia              | Capturas/video del flujo y export de evidencia técnica.                    |

### DEFINICIÓN DE CIERRE VISUAL

FASE 1 se vuelve “visible” cuando un tercero no técnico puede abrir la demo, crear un compromiso, ver qué decisión ocurre, observar la ejecución/reconciliación y confirmar el resultado final con evidencia, sin leer código ni ejecutar SQL.

## 9. BLOQUEOS Y RIESGOS QUE DEBEN SEGUIR VISIBLES

| Riesgo            | Estado                                                                                                                     | Acción                                                                                       |
|-------------------|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| GitHub Actions    | Runs recientes mueren antes de Checkout/steps; no es un test fallido, pero impide usar CI verde como evidencia del commit. | Resolver runner/Actions o ejecutar certificación formal fuera de GitHub y guardar evidencia. |
| UI inexistente    | El motor no tiene frontend operativo certificado.                                                                          | Crear demo UI mínima; no confundir con dashboard productivo completo.                        |
| Deployment        | No hay entorno comercial declarado.                                                                                        | Staging primero; producción después de certificación y operaciones.                          |
| Provider real     | La demo puede usar provider simulado; cada integración real requiere contrato, credenciales y pruebas propias.             | Implementar/adaptar provider por caso de uso.                                                |
| Reliability score | DC y score compuesto no están definidos por la fuente.                                                                     | Mantenerlos UNAVAILABLE hasta aprobar fórmula versionada.                                    |
| Performance DB    | Existen recomendaciones de performance (índices/políticas RLS), no lints de seguridad críticos.                            | Hardening posterior; no confundir con fallo funcional.                                       |

## 10. CONCLUSIÓN - DÓNDE ESTAMOS REALMENTE

### CONCLUSIÓN TÉCNICA

NEXUS COMMIT no es “nada”: existe un motor considerable y coherente, con ejecución, reconciliación, persistencia, autoridad, RLS, workers, outbox, readiness y certificación. Parte de la cadena central ya fue ejecutada contra PostgreSQL real y terminó en SUCCEEDED. El problema es de presentación/productización: ese motor todavía no está empaquetado como una aplicación visible que usted pueda abrir y manejar.

### CONCLUSIÓN DE PRODUCTO

Para poder sentarse frente a un inversionista y “mostrar NEXUS COMMIT”, el siguiente objetivo no debe ser agregar más arquitectura. Debe ser construir una demo visible end-to-end sobre el motor ya existente y cerrar una certificación del commit exacto.

## ESTADO DE FASE 1: CORE FUNCIONAL IMPLEMENTADO + DEMOSTRACIÓN VISIBLE PENDIENTE

Corte oficial: ab340f75f6a0634e4f93462b71a152bdb640a541

## ANEXO A. CIERRES RECIENTES QUE CONSTRUYERON ESTE ESTADO

- PR #7 - corrección del gate EXECUTION\_PENDING en dispatch.
- PR #8 - guard de provider ausente durante reconciliación.
- PR #9 - tests alineados con provider authority runtime.
- PR #10 - fail-closed si PARTIAL/TIMEOUT no tiene execution profile durable.
- PR #11 - demo E2E execution/reconciliation.
- PR #12 - envelopes completos para persistencia de eventos de ejecución.
- PR #13 - demo PostgreSQL reproducible.
- PR #14 - ruta completa runtime PostgreSQL con workers y journal.
- PR #15 - evidencia de provider\_execution\_id en STATUS\_QUERY.
- PR #16 - prueba Outbox 3/3 delivered.
- PR #17 - replay safety / segunda pasada = 0.
- PR #18 - runtime proof obligatorio dentro de pnpm certify.
- PR #19 - aislamiento concurrente de certificaciones.
- PR #20 - readiness exige migración 038.
- PR #21 - readiness API + worker probado en runtime demo.
- PR #22 - health de outbox alineado a claimability.
- PR #23 - OUTBOX\_FAILED visible en health.
- PR #24 - DEADLINE\_FAILED visible en health.
- PR #25 - documentación de deployment alineada a schema/certificación.
- PR #26 - cobertura completa de operational health guards.
- PR #27 - certificación exige 0 violaciones operacionales.
- PR #28 - hardening de configuración de endpoints de providers.
- PR #29 - proof de atomicidad/rollback de persistencia.
- PR #30 - persistence integration activo durante certificación.
- PR #31 - 12/12 integraciones declaradas + CI delega a pnpm certify.
- PR #32 - fase unit/static de CI sin DB; DB queda en certificación formal.

Nota: este documento describe el corte oficial en main. Un hardening posterior de redacción de errores internos de API quedó fuera de este corte y no se contabiliza como terminado.